

CSE398/498 Lab 1: Object Detection with YOLO11 and YOLOE

Xiaoxi Gao | Fall 2026

1. Goal of the Lab

I used this lab to compare three ways of doing object detection on the same set of objects: (1) a pretrained YOLO11 model, (2) a fine-tuned YOLO11 model, and (3) YOLOE with open-set prompts. I kept the same 15 test images for the main comparison so that the difference mainly came from the model or prompt, not from changing the test set.

2. How YOLO11 Works

YOLO11 is a one-stage detector. An input image goes through a backbone that extracts visual features, a neck that combines features from different scales, and a detection head that predicts object locations and class scores. Because detection happens in one network pass, YOLO is fast enough for webcam-style use.

For this lab I used YOLO11n. The model returns bounding boxes, predicted classes, and confidence scores. I used a confidence threshold of 0.50 for the baseline and fine-tuned image tests.

3. Test Set

I captured 15 images from my own environment. The ground-truth classes were: garlic (test01-02), apple (test03-07), key (test08-09), game controller (test10), glasses (test11-12), and green chili (test13-15).

Class	Images	Count
garlic	test01-test02	2
apple	test03-test07	5
key	test08-test09	2
game_controller	test10	1
glasses	test11-test12	2
green_chili	test13-test15	3

4. Experiment 1 - Pretrained YOLO11 Baseline

I first ran the original pretrained YOLO11n model without any retraining. The script saved both the raw image and the annotated result, and also wrote the detected class and confidence to a CSV file.

The baseline result was weak on my six target classes. Only 3 of the 15 images had the correct top prediction, which gives a simple image-level accuracy of 20%.

Image(s)	Ground truth	Typical baseline result
test01-02	garlic	apple
test03-07	apple	apple / miss / orange
test08-09	key	scissors / knife
test10	game_controller	remote
test11-12	glasses	scissors
test13-15	green_chili	banana / person

What I noticed

The wrong predictions were understandable from the appearance of the objects. The garlic bulb is round and was often treated like fruit. The key has a thin metallic shape and was confused with knife or scissors. The controller was predicted as a remote. The green chili images were especially difficult because the object shape, lighting, and background were different from common pretrained examples.

This baseline gave me a useful starting point because it showed exactly which classes needed task-specific training.

5. Baseline Result

Correct: 3/15

Wrong or missed: 12/15

Image-level top-1 accuracy: 20%

I use this 20% number only as a simple comparison on my fixed 15-image set. It is not an object-detection mAP score.

6. Experiment 2 - Fine-tuning YOLO11

For fine-tuning, I built a Roboflow object-detection dataset with six classes: apple, game_controller, garlic, glasses, green_chili, and key. I collected many of the images myself, and for classes that were not training well I deliberately added more views from different angles, distances, and backgrounds. Following Prof. Chuah's revised instructions, every image had a bounding-box ground-truth annotation and the dataset was exported in YOLO format.

The final Roboflow v3 dataset contained 203 images after preprocessing and augmentation. I used Auto-Orient, resized images to 640 x 640 with Stretch, and applied horizontal flipping, brightness variation ($\pm 15\%$), and exposure variation ($\pm 10\%$), with three outputs per training example. I initially organized the source data with an approximately 7:2:1 train/validation/test target split. Because augmentation was applied mainly to the training portion, the final exported v3 dataset contained 177 training images, 18 validation images, and 8 test images (87%/9%/4%). I then trained YOLO11n for 50 epochs on a T4 GPU.

Final validation results

Metric	Final value
Precision	0.824
Recall	0.872
mAP@0.50	0.899
mAP@0.50:0.95	0.573

The final validation of best.pt used 18 validation images and achieved Precision = 0.824, Recall = 0.872, mAP@0.50 = 0.899, and mAP@0.50:0.95 = 0.573. Green_chili performed strongly after I added more varied real examples (Recall = 1.0 and mAP@0.50 = 0.995), while key remained the weakest validation class. This reinforced what I saw during data collection: adding genuinely different views of a weak object was more useful than only repeating similar images.

7. Re-testing the Same 15 Images

I then used the final best.pt checkpoint on exactly the same 15 raw images used in the baseline experiment. None of these test01-test15 images was included in the 203-image fine-tuning dataset, so the re-test did not reuse training images. On this held-out 15-image set, all 15 images were predicted with the correct target class.

Class	Fine-tuned confidence range / result
garlic	0.89-0.92
apple	0.91-0.99
key	0.96-0.99
game_controller	0.58
glasses	0.78-0.96
green_chili	0.81-0.94

Baseline: 3/15 correct (20%)

Fine-tuned: 15/15 correct (100%)

Improvement on this fixed test set: +80 percentage points

8. What I Learned from Fine-tuning

The biggest lesson was that fine-tuning did more than increase confidence. It changed the class decision itself. For example, garlic changed from apple to garlic, the controller changed from remote to game_controller, and the green chilies changed from banana/person or misses to green_chili.

I also saw how important the dataset is. My first green-chili result was poor even though the training code was working correctly. The problem was the data coverage. After I added more real green-chili examples with different shapes and views, the class improved a lot.

The final 15/15 result is strong for this lab setup, but I would not call it universal 100% accuracy. The test set is small, and it is related to the same task and visual setting used when I built the dataset. A larger independent test set would be needed before making a broader claim.

9. Why Fine-tuning Helped

- The model received explicit examples of the six target labels.
- Bounding boxes taught the detector both what the object was and where it was.
- More real variation improved a weak class better than relying only on augmentation.
- Using the same 15 images before and after training made the improvement easy to see.

10. Open-set Object Detection

YOLO11 is a closed-set detector in this experiment because its normal class vocabulary is fixed by training. YOLOE is different: at inference time I can give it a text prompt or a visual example and ask it to look for that concept without doing a new fine-tuning run.

This is useful when the class list can change. The trade-off I observed is that open-set prompting is flexible, but it can be sensitive to the exact prompt and can produce extra competing detections.

11. Experiment 3 - YOLOE Text Prompt

I ran YOLOE on the same 15 images with a confidence threshold of 0.10. My first text-prompt set used short descriptions such as "red apple," "game controller," "whole garlic bulb," "black eyeglasses," "green chili pepper," and "metal key."

With this first prompt set, the top-confidence class was correct on 9 of 15 images (60%). Apple and garlic worked fairly well, but key, game controller, and green chili were much less stable.

Prompt refinement

I then rewrote each prompt using visible details from the objects, for example "round purple garlic bulb with striped skin," "red video game controller with two joysticks," and "small curved wrinkled green chili pepper on a wooden table." The final prompt version reached 12/15 correct top predictions (80%).

YOLOE text prompt version	Top-1 result
Initial prompts	9/15 = 60%
Final descriptive prompts	12/15 = 80%
Change	+20 percentage points

The final prompts were better overall, but they also created more false positives. For example, test10 correctly detected the game controller at 0.68, but also produced a key detection at 0.66. So I stopped tuning at this point instead of trying to force 15/15 by repeatedly adjusting the test prompts.

12. YOLOE Visual Prompt

For difficult cases, I used the notebook's visual-prompt workflow. I drew a bounding box around a reference object, YOLOE created a visual prompt embedding, and I reused that embedding on a different target image.

- Green chili: a visual reference recovered test15 as green_chili with confidence 0.3021.
- Key: test08 was used as the reference, and test09 was detected as key with confidence 0.1200.

These confidence values were not high, but both examples passed the 0.10 threshold and showed that a visual reference could recover objects that text prompting handled poorly.

13. YOLOE Video Demos

I also completed both video sections in the provided YOLOE notebook. For the text-prompt video demo, I used the suitcase example and YOLOE detected suitcases frame by frame. For the visual-prompt video demo, I used the croissant example: I selected the object in a reference frame, generated the visual prompt embedding, and used it across the video.

These video runs were supplementary notebook exercises. I did not include them in the 15-image accuracy numbers because they used the notebook's example videos rather than my fixed image test set.

14. Overall Comparison

Method	Main result on 15 images	Main observation
Pretrained YOLO11	3/15 = 20%	Fast, but poor match for my target classes
Fine-tuned YOLO11	15/15 = 100%	Best result for this fixed six-class task
YOLOE initial text	9/15 = 60%	Flexible but prompt-sensitive
YOLOE final text	12/15 = 80%	Better prompts helped, but extra false positives remained
YOLOE visual prompt	Recovered key/chili cases	Useful when text description was weak

15. Conclusion

The three experiments gave me a clear comparison. The pretrained YOLO11 model was not reliable for several of my chosen objects. Fine-tuning with labeled examples gave the strongest result on this specific task. YOLOE was less accurate than the fine-tuned detector on the 15-image test, but it could add new concepts at inference time without another training run.

For me, the most useful takeaway is that the right approach depends on the situation. If the target classes are known and labeled data is available, fine-tuning is the better choice. If the class list can change or there is no time to retrain, YOLOE text or visual prompts provide a practical alternative.

References

- [1] CSE398/498 Lab Assignment 1: Object Detection with YOLO11 and YOLOE, Fall 2026.
- [2] Ultralytics YOLO documentation and YOLO11 training/inference tools used in the lab.
- [3] YOLOE: Real-Time Seeing Anything, 2025.
- [4] Roboflow / Ultralytics Colab notebooks used for YOLO11 fine-tuning and YOLOE prompting.